

重庆邮电大学

学生实验实习报告册

学年学期： 2024-2025 学年 ☐春☒秋学期

课 程 名 称： 信号处理实验

学 生 学 院： 通信与信息工程学院

专 业 班 级： 01012205

学 生 学 号： 2022210410

学 生 姓 名： 雷宇航

联 系 电 话： 18580086873

重庆邮电大学教务处制

课程名称	信号处理实验	课程编号	A2010550
实验地点	YF309	实验时间	2024. 10. 22 周二（3-4）节
校外指导教师	无	校内指导教师	郑丹玲
实验名称	系统响应与系统稳定性		
评阅人签字		成绩	
一、实验目的 （1）学会运用 Matlab 求解离散时间系统的零状态响应； （2）学会运用 Matlab 求解离散时间系统的单位取样响应； （3）学会运用 Matlab 求解离散时间系统的卷积和。 二、实验原理 1. 离散时间系统的响应 离散时间 LTI 系统可用线性常系数差分方程来描述，即 $\sum_{i=0}^N a_i y(n-i) = \sum_{j=0}^M b_j x(n-j) \quad (1)$ 其中，a_i（$i=0,1,\dots,N$）和 b_j（$j=0,1,\dots,M$）为实常数。 对于类似式(1)的离散时间系统的零状态响应求解，本实验使用 Matlab 中函数 filter 对相应的差分方程，对在指定时间范围内的输入序列所产生的响应进行求解。函数 filter 的语句格式为 $y = \text{filter}(b,a,x)$ 其中，x为输入的离散序列；y为输出的离散序列；y的长度与x的长度一样；b与a分别为差分方程右端与左端的系数向量。 2. 离散时间系统的单位取样响应 对于离散时间系统的单位取样响应的求解，系统的单位取样响应定义为系统在$\delta(n)$激励下系统的零状态响应，用$h(n)$表示。Matlab 求解单位取样响应可利用函数 filter，并将激励设为单位抽样序列。 Matlab 另一种求单位取样响应的方法是利用控制系统工具箱提供的函数 impz 来实现。impz 函数的常用语句格式为 $\text{impz}(b,a,N)$ 其中，参数 N 通常为正整数，代表计算单位取样响应的样值个数。 3. 离散时间信号的卷积和运算 由于系统的零状态响应是激励与系统的单位取样响应的卷积，因此卷积运算在离散时间信号处理领域被广泛应用。离散时间信号的卷积定义为			

$$y(n) = x(n) * h(n) = \sum_{m=-\infty}^{\infty} x(m)h(n-m) \quad (2)$$

可见，离散时间信号的卷积运算是求和运算，因而常称为“卷积和”。

Matlab 求离散时间信号卷积和的命令为 `conv`，其语句格式为

$$y = \text{conv}(x, h)$$

其中， x 与 h 表示离散时间信号值的向量； y 为卷积结果。用 Matlab 进行卷积和运算时，无法实现无限的累加，只能计算时限信号的卷积。

4. Matlab 中的二维卷积函数 `conv2()`

(1) `C=conv2(A,B)`

设矩阵 A 的大小为 $M_a \times N_a$ ，矩阵 B 的大小为卷积 $M_b \times N_b$ ，则卷积后 C 的大小为 $(M_a + M_b - 1) \times (N_a + N_b - 1)$ ；

(2) `C=conv2(Hcol,Hrow,A)`

矩阵 A 先与 $Hcol$ 向量在列方向进行卷积，然后再与 $Hrow$ 向量在行方向上进行卷积；

(3) `C=conv2(A,B,'shape')`

当 `shape=full` 时，返回全部二维卷积结果，即返回 C 的大小为 $(M_a + M_b - 1) \times (N_a + N_b - 1)$ ；

当 `shape=same` 时，返回与 A 同样大小的卷积中心部分；

当 `shape=valid` 时，不考虑边界补零，即只要有边界补出的零参与运算的都舍去，返回 C 的大小为 $(M_a + M_b - 1) \times (N_a + N_b - 1)$ ；

`conv2` 函数常用于对图像进行滤波处理。

5. 单位阶跃序列

单位阶跃序列 $u(n)$ 定义为

$$u(n) = \begin{cases} 1 & (n \geq 0) \\ 0 & (n < 0) \end{cases}$$

在 Matlab 中，冲激序列可以通过编写 `uDT.m` 文件来实现，即

```
1. function y=uDT(n)
2. y=n>=0; %当参数为非负时输出 1
```

调用该函数时 n 也同样必须为整数或整数向量。

三、实验程序及结果分析

(1) 试用 Matlab 命令求解以下离散时间系统的单位取样响应，并判断系统的稳定性。

$$1) \quad 3y(n) + 4y(n-1) + y(n-2) = x(n) + x(n-1)$$

代码:

```
1. %% 问题一
2. %% 清空
3. clear all;
4. clc;
5. %% 系统 1 采用 filter 求单位冲击响应 并做零极点图
6. a1=[3 4 1];
7. b1=[1 1];
8. n1=0:29;
9. x=(n1==0);
10. h1=filter(b1,a1,x);
11. figure(1);
12. subplot(2,1,1);
13. stem(n1,h1,'fill','r');
14. grid on;
15. xlabel('n');
16. ylabel('h_{1}(n)');
17. title('单位取样响应 h_{1}(n)');
18. subplot(2,1,2);
19. zplane(b1,a1);
20. grid on;
21. legend('零点','极点');
22. title('H_{1}(z)零极点分布图');
```

输出:

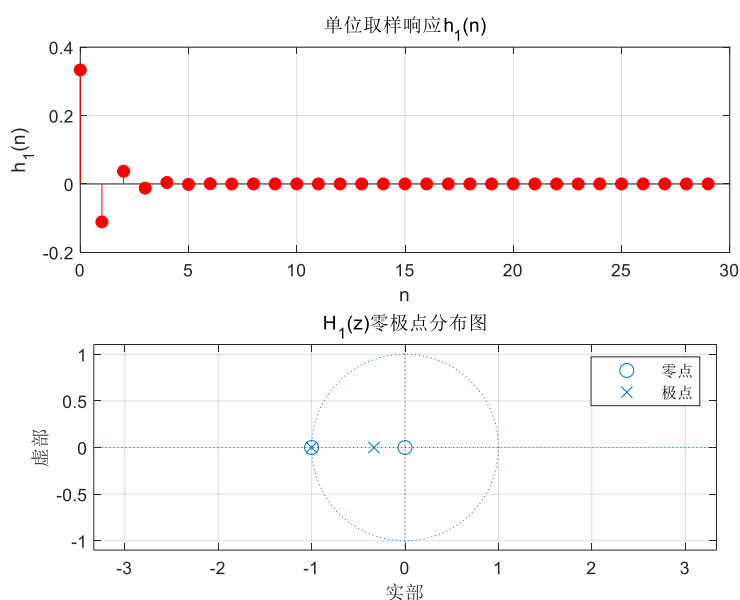


图 1 系统 1 的单位取样响应 $h_1(n)$ 和零极点图

分析:

由上面的波形图可以看出,随着 n 的增大, $h_1(n)$ 逐渐收敛于 0, 所以可以判断出系统 1 是稳

定的。

在离散时间因果系统中:当有极点出现在单位圆外部时,或者在单位圆内部出现重极点,则系统不稳定;如果除了在单位圆上有单阶极点,其他极点都在单位圆内部,则系统临界稳定;若极点均在单位圆内部,则系统稳定。

分析零极图可知系统 1 在单位圆上有单阶极点,但该极点与零点相互抵消,所以极点在单位圆内部,系统稳定。

$$2) \quad \frac{5}{2}y(n) + 6y(n-1) + 10y(n-2) = x(n)$$

代码:

```
1. %% 问题一
2. %% 清空
3. clear all;
4. clc;
5. %% 系统 2 采用 impz 求单位冲击响应 并做零极图
6. a2=[5/2 6 10];
7. b2=[1];
8. n2=30;
9. figure(2);
10.subplot(2,1,1);
11.impz(b2,a2,n2);
12.grid on;
13.xlabel('n');
14.ylabel('h_{2}(n)');
15.title('单位取样响应 h_{2}(n)');
16.subplot(2,1,2);
17.zplane(b2,a2);
18.grid on;
19.legend('零点','极点');
20.title('H_{2}(z)零极点分布图');
```

输出:

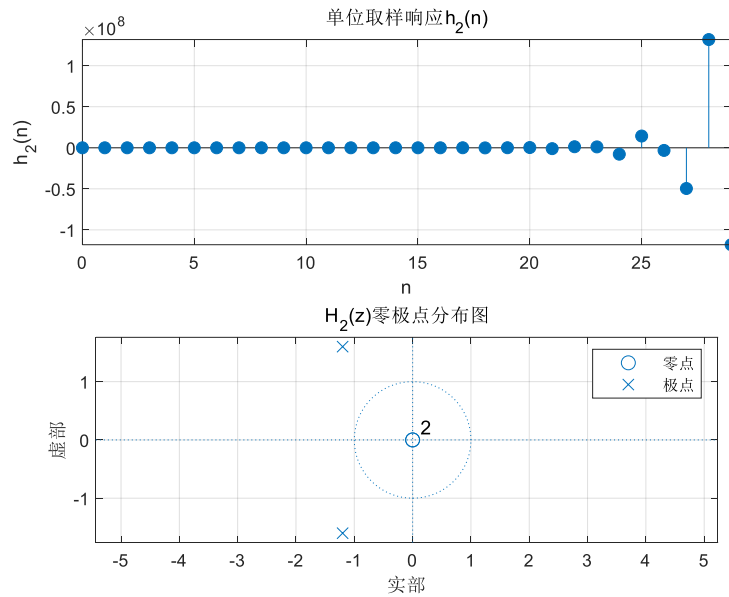


图 2 系统 2 的单位取样响应 $h_2(n)$ 和零极点图

分析:

由上面的波形图可以看出, 随着 n 的增大, $h_2(n)$ 逐渐发散, 所以可以判断出系统 2 不稳定。

分析零极点图可知系统 2 极点均位于单位圆外, 系统不稳定。

(2) 已知某系统函数的单位取样响应为 $h(n) = \left(\frac{7}{8}\right)^n [u(n) - u(n-10)]$, 试用 Matlab 求当激励信

号为 $x(n) = u(n) - u(n-5)$ 时, 系统的零状态响应。

代码:

```
1. %% 问题二
2. %% 清空
3. clear all;
4. clc;
5. %% 卷积和
6. nh=-4:20;
7. nx=-4:16;
8. hn=(7/8).^nh.*(heaviside(nh)-heaviside(nh-10));
9. xn=heaviside(nx)-heaviside(nx-5);
10. yn=conv(xn,hn);
11. ny1=nx(1)+nh(1);
12. ny2=nx(end)+nh(end);
13. ny=ny1:ny2;
14. %% 显示
15. figure;
16. subplot(3,1,1);
17. stem(nx,xn,'fill','r','LineWidth',0.6);
18. grid on;
```

```

19.axis([-4,16,0,2]);
20.xlabel('n');
21.title('x_{n}');
22.subplot(3,1,2);
23.stem(nh,hn,'fill','g','LineWidth',0.6);
24.grid on;
25.axis([-4,16,0,2]);
26.xlabel('n');
27.title('h_{n}');
28.subplot(3,1,3);
29.stem(ny,yn,'fill','b','LineWidth',0.6);
30.grid on;
31.axis([-4,16,0,5.5]);
32.xlabel('n');
33.title('y_{n}=x_{n}*h_{n}');

```

输出:

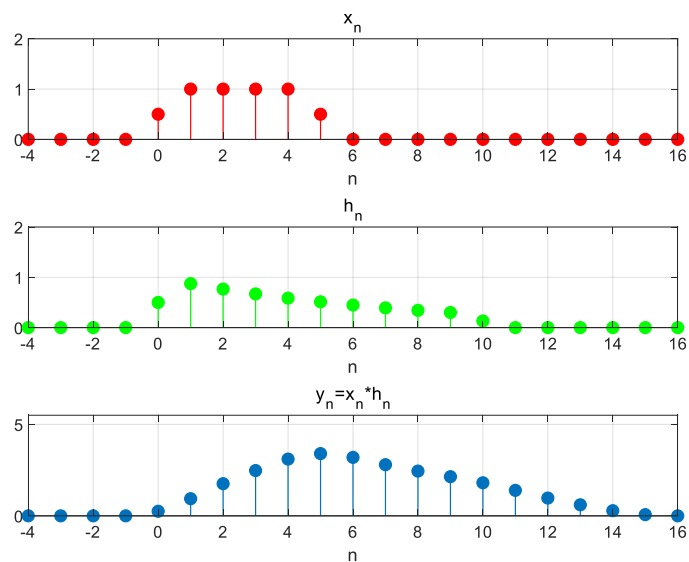


图3 系统零状态响应 $y(n)$

分析:

由题意可知，描述 $h(n)$ 向量长度至少为 10，描述 $x(n)$ 向量长度至少为 5，描述两个离散序列的卷积和长度至少为 14，因此为了图形完整美观，我们将 $h(n)$ 向量和 $x(n)$ 向量加上一些附加的零值，通过 Matlab 作图得出卷积和长度为 14。

(3) 用数字图像处理的 Sobel 算子，调用 `conv2` 函数对 `lena` 图像进行滤波。两个 3×3 的 Sobel 矩阵算子 $G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ ， $G_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$ 。

- 1) 读取 `lena` 图像数据，保存到矩阵 `B` 中；
- 2) 分如下三种情况对 `B` 进行卷积滤波
 - ① 采用 G_x 与 `B` 进行卷积；
 - ② 采用 G_y 与 `B` 进行卷积；
 - ③ 先采用 G_x 与 `B` 进行卷积，然后再采用 G_y 与 `B` 进行卷积；
 - ④ 在一个 `figure` 中，分四个子图显示出：原始图像，经过 G_x 卷积滤波后图像，经过 G_y 卷积滤波后图像，先后采用 G_x 和 G_y 与 `B` 进行卷积滤波后的图像，观察滤波后图像的效果。

代码：

```
1. %% 问题三
2. %% 清空
3. clear all;
4. clc
5. %% 定义 Sobel 算子
6. Gx=[-1 0 1;-2 0 2; -1 0 1];
7. Gy=[1 2 1;0 0 0;-1 -2 -1];
8. %% 读取 lena 图像
9. B=imread('lena.bmp');
10. F1=conv2(Gx,B);
11. F2=conv2(Gy,B);
12. F3=conv2(F1,Gy);
13. F1=uint8(F1);
14. F2=uint8(F2);
15. F3=uint8(F3);
16. %% 图像显示
17. figure;
18. subplot(2,2,1);
19. imshow(B);
20. title('原始图像');
21. subplot(2,2,2);
22. imshow(F1);
23. title('Gx 卷积滤波图像')
24. subplot(2,2,3);
25. imshow(F2);
26. title('Gy 卷积滤波图像')
27. subplot(2,2,4);
28. imshow(F3);
29. title('先后采用 Gx,Gy 卷积滤波图像')
```


输出:

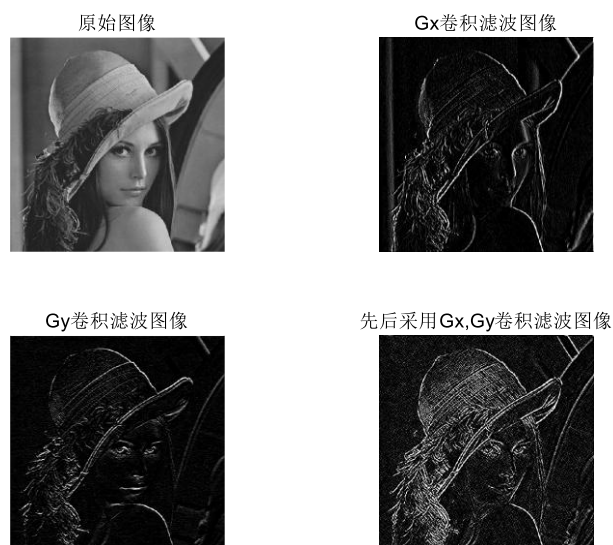


图 4 Sobel 算子边缘检测结果图

分析:

Sobel 算子对灰度渐变和噪声较多的图像处理效果比较好, Sobel 算子对边缘定位比较准确。Gx 用于计算横向的梯度和检测纵向边缘, Gy 用于计算纵向的梯度和检测横向边缘。先后进行两次卷积, 可以较好的提取出图像整体的边缘。

四、思考题

(1) Matlab 的工具箱函数 `conv`,能用于计算两个有限长序列的卷积, 但 `conv` 函数假定这两个序列都从 $n=0$ 开始。编写 M 文件计算 $x(n)=[3,11,7,0,-1,4,2], -3 \leq n \leq 3$ 和 $h(n)=[2,3,0,-5,2,1], -1 \leq n \leq 4$ 的卷积, 并绘制 $y(n)$ 的波形图。

代码:

```
1. %% 思考题一
2. %% 清空
3. clear all;
4. clc;
5. %% 卷积
6. nx=-3:3;
7. nh=-1:4;
8. xn=[3,11,7,0,-1,4,2];
9. hn=[2,3,0,-5,2,1];
10. yn=conv(xn,hn);
11. ny1=nx(1)+nh(1);
12. ny2=nx(end)+nh(end);
13. ny=ny1:ny2;
14. %% 作图
```

```

15.figure;
16.stem(ny,yn,'fill','k','Linewidth',0.6);
17.xlabel('n');
18.ylabel('y_{n}');
19.title('y_{n}=x_{n}*h_{n}')
20.grid on;

```

输出:

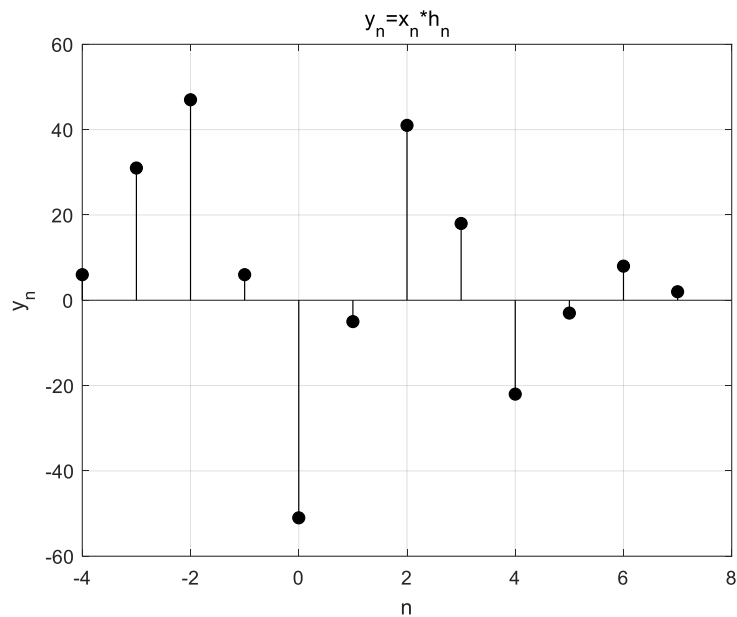


图5 有限长序列卷积和

分析:

两个有限长序列的卷积和通过调用 `conv` 函数求解出序列的值，但结果的范围需要根据两个序列的范围去确定，起始位置和结束位置分为两个有限长序列起始位置和结束位置的叠加，从而确认了卷积和的范围。

(2) 在数字图像处理边缘检测技术中，除了 Sobel 算子外，还有哪些边缘检测？通过查找资料，选择某种边缘检测算子，在 Matlab 平台上编程实现对 lena 或其他图像进行边缘检测，显示出原图和边缘检测后的图像。

代码:

```

1. %% 思考题二
2. %% 清空
3. clear all;
4. clc;
5. %% 图像读取
6. cat=imread('cat.jpg');
7. GrayCat=rgb2gray(cat);
8. %% Canny 边缘检测
9. F1=edge(GrayCat,'Canny');

```

```

10.%% Prewitt 边缘检测
11.F2=edge(GrayCat,'Prewitt');
12.%% log 边缘检测
13.F3=edge(GrayCat,'log');
14.%% Roberts 边缘检测
15.F4=edge(GrayCat,'Roberts');
16.%% 图像显示
17.figure;
18.subplot(1,2,1);
19.imshow(cat);
20.title('原始图像')
21.subplot(1,2,2);
22.imshow(GrayCat);
23.title('原始灰度图像');
24.figure;
25.subplot(2,2,1);
26.imshow(F1);
27.title('Canny 边缘检测图像')
28.subplot(2,2,2);
29.imshow(F2);
30.title('Prewitt 边缘检测图像')
31.subplot(2,2,3);
32.imshow(F3);
33.title('log 边缘检测图像')
34.subplot(2,2,4);
35.imshow(F4);
36.title('Roberts 边缘检测图像')

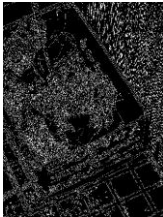
```

输出:

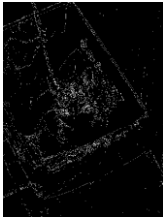


图6 彩色图和灰度图显示

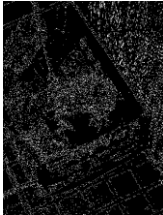
Canny边缘检测图像



Prewitt边缘检测图像



log边缘检测图像



Roberts边缘检测图像

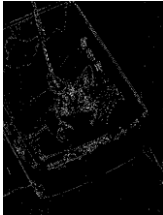


图 7 四种边缘检测结果图

分析：

根据上图总结出下表规律：

表 1 四种边缘检测算子对比

算子	边缘检测效果	噪声抑制能力	边缘定位准确性
Canny	较为清晰，能够检测到 细微边缘	较好	高
Prewitt	边缘较为明显，但可能 出现锯齿状	一般	中等
LoG	边缘清晰，能够捕捉到 较宽的边缘变化	较好	较高
Roberts	边缘较为清晰，但可能 会受到噪声影响	一般	中等